



**NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY**

Intrusion Analysis and Response

IAR721S

Group Assignment 2026

Network Intrusion Detection and Response

Group Members

Titus Shaangelo 222084774 (Group leader)

Stella Apollus 222045868

Brnabas Iihuhwa 223081701

Uetusuvera Tjivau 224077031

Introduction

In an era of increasingly sophisticated cyberthreats, financial institutions are primary targets for malicious actors due to the high value of sensitive financial records. Traditional perimeter defences are no longer sufficient to stop modern, multi-stage attacks. This project demonstrates the implementation of a Network Intrusion Detection System (NIDS) and host-based security measures to protect a simulated financial workstation. By deploying Snort as an IDS and utilizing tools like Kali Linux for offensive simulation, this project provides a holistic view of the "Detect, Analyse, and Respond" lifecycle required to maintain the integrity and confidentiality of critical digital infrastructure.

Attack Methodology and Tools

The attack simulation follows a multi-stage "Kill Chain" methodology designed to mirror a real-world breach on a financial target.

The Attack Chain:

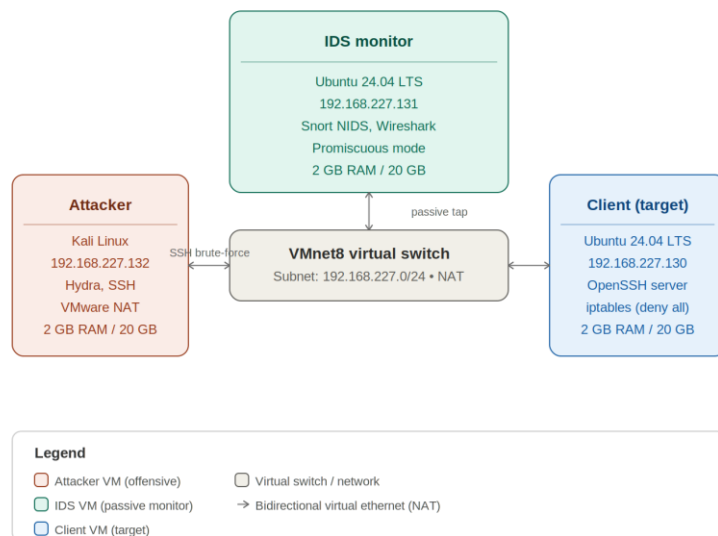
1. **Reconnaissance & Brute Force:** The Attacker uses automated tools to identify the SSH service and attempt to crack user credentials.
2. **Exploitation:** Once credentials are recovered, the Attacker gains unauthorized shell access to the target system.
3. **Privilege Escalation:** The Attacker attempts to elevate privileges to the 'root' user to gain full control over the workstation.
4. **Data Exfiltration:** The Attacker locates the financial_records.txt file and transfers it to the remote Kali machine.

Software and Tools Used:

- **Kali Linux:** The offensive platform used for launching Hydra and SCP.
- **Ubuntu Desktop (Client):** The victim machine, hardened using iptables.
- **Ubuntu (IDS):** The monitoring hub running **Snort** (Network Intrusion Detection) and **Wireshark** (Packet Analysis).
- **Hydra:** Used for high-speed SSH brute-forcing.
- **SCP (Secure Copy):** Used to simulate the exfiltration of sensitive data.

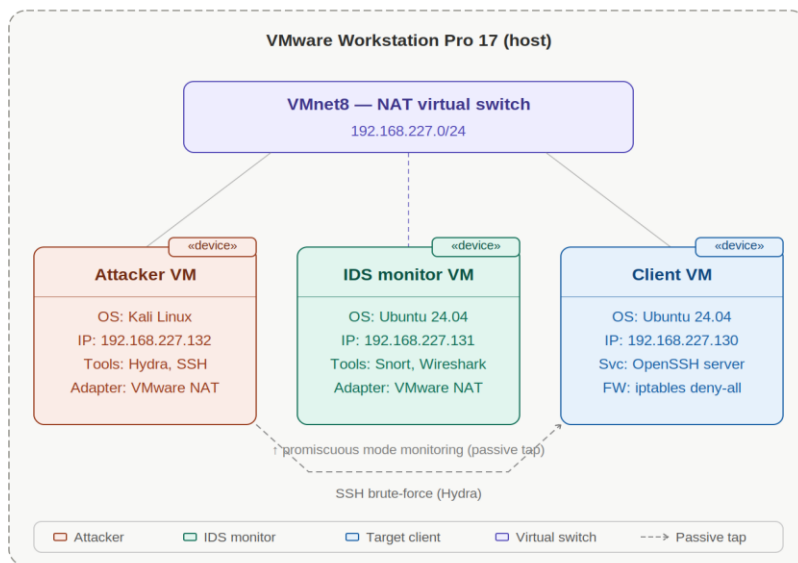
Network Topology Overview

This diagram shows the three virtual machines connected through the VMnet8 NAT virtual switch. The Attacker VM (Kali Linux) launches SSH brute-force attacks via Hydra against the Client VM (Ubuntu). The IDS Monitor VM (Ubuntu) passively captures all traffic using Snort NIDS and Wireshark in promiscuous mode.



UML Deployment Diagram

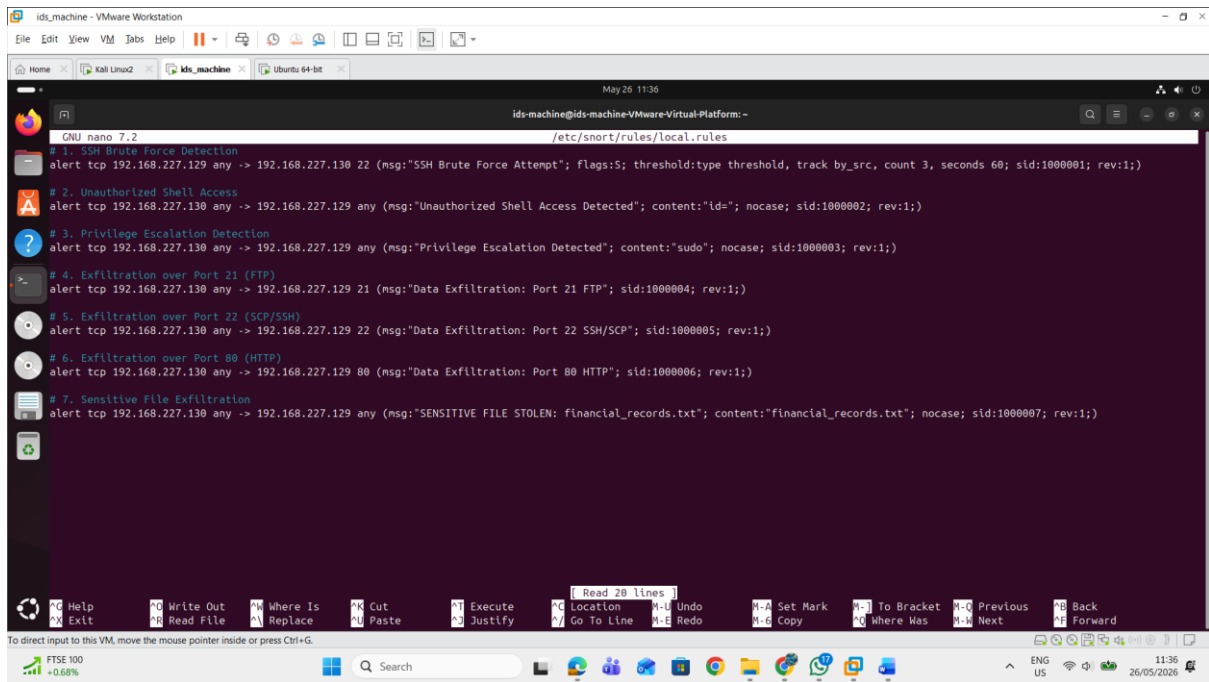
This UML deployment diagram uses standard «device» stereotypes to represent each virtual machine as a deployment node within the VMware Workstation Pro 17 host boundary. The dashed border indicates the execution environment, while the dashed connector to the IDS Monitor denotes its passive promiscuous monitoring role. The SSH brute-force traffic path from Attacker to Client is shown as a dashed arc below the VMs.



IDS Configuration and Rule Logic (Snort)

A custom ruleset was developed for Snort to monitor the internal network traffic for indicators of compromise (IoC). The following table explains the technical logic and compliance of each rule:

# Rule	Detection	Purpose
1 SSH Brute Force	3 SYN attempts to TCP/22 within 60s	Detect repeated SSH login attempts
2 Unauthorized Shell Access	Payload contains id=	Detect attacker shell command execution
3 Privilege Escalation	Payload contains sudo	Detect sudo usage after compromise
4 FTP Exfiltration	Outbound TCP/21 traffic	Detect FTP-based data theft
5 SSH/SCP Exfiltration	Outbound TCP/22 traffic	Detect SCP/SSH file exfiltration
6 HTTP Exfiltration	Outbound TCP/80 traffic	Detect HTTP/Netcat exfiltration
7 Sensitive File Exfiltration	Payload contains financial_records.txt	Detect theft of sensitive file

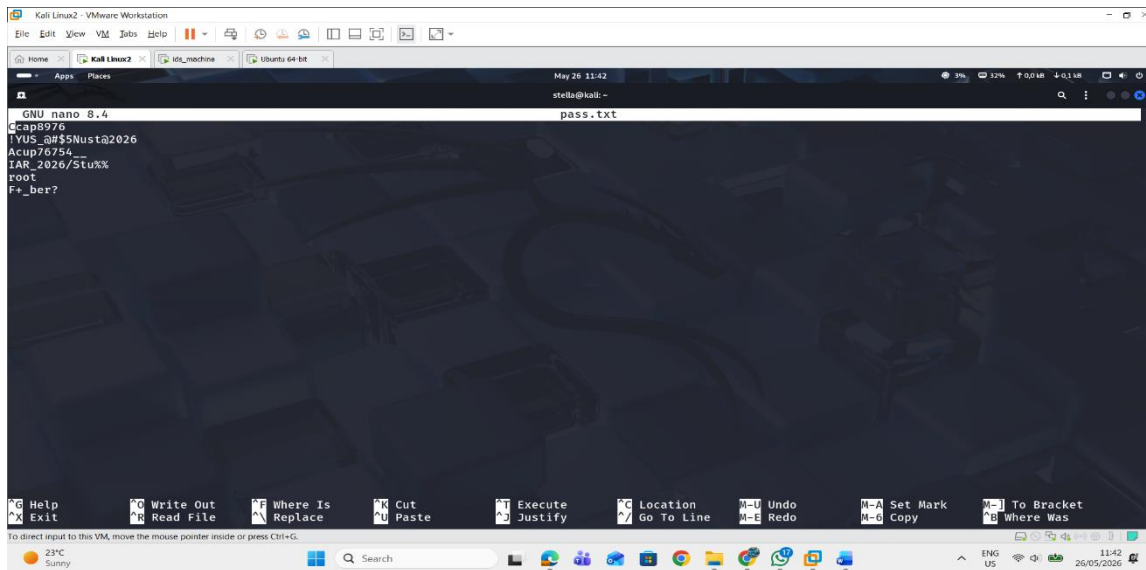


Weak Password Simulation for Attack Testing

For the purpose of this simulation, the Client Workstation was intentionally configured with a weak, common password ('root'). This simulates a 'Vulnerable System' within a financial organization, allowing us to demonstrate how quickly a brute-force attack can succeed when basic password policies are not enforced. In contrast, the IDS machine uses a more complex password ('Student123!') to reflect a more secure administrative environment.

Creating a Manual Password Wordlist in Kali Linux

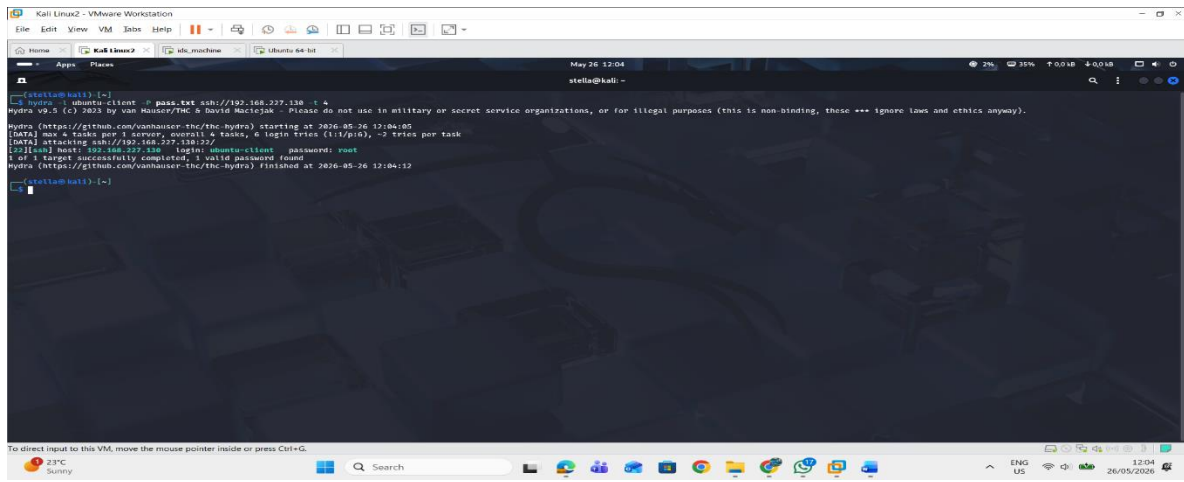
A custom password wordlist was manually created in Kali Linux to improve the efficiency and speed of the password-cracking simulation. Instead of using the large default rockyou.txt wordlist, which requires extraction and contains millions of passwords, a smaller tailored wordlist was used containing only a few likely passwords, including the correct login credential for the Ubuntu client machine. This approach reduced attack execution time, simplified the testing process, lowered system resource usage, and made the simulation easier to document and demonstrate during the presentation and IDS monitoring phase.



SSH Brute-Force Attack: Execution and IDS Detection

To gain initial access to the Client VM, a dictionary-based SSH brute-force attack was launched from the Attacker VM using THC-Hydra with the command `hydra -l ubuntu-client -P pass.txt ssh://192.168.227.130 -t 4 -V`. The `-l` flag specified the target username, `-P pass.txt` provided the password wordlist, `ssh://192.168.227.130` defined the SSH service and target IP, `-t 4` enabled four parallel threads to speed up the attack, and `-V` displayed each login attempt in real time. As shown in Figure 3.3.2a, valid credentials (ubuntu-client: root) were successfully recovered within seconds, demonstrating the risk of weak passwords.

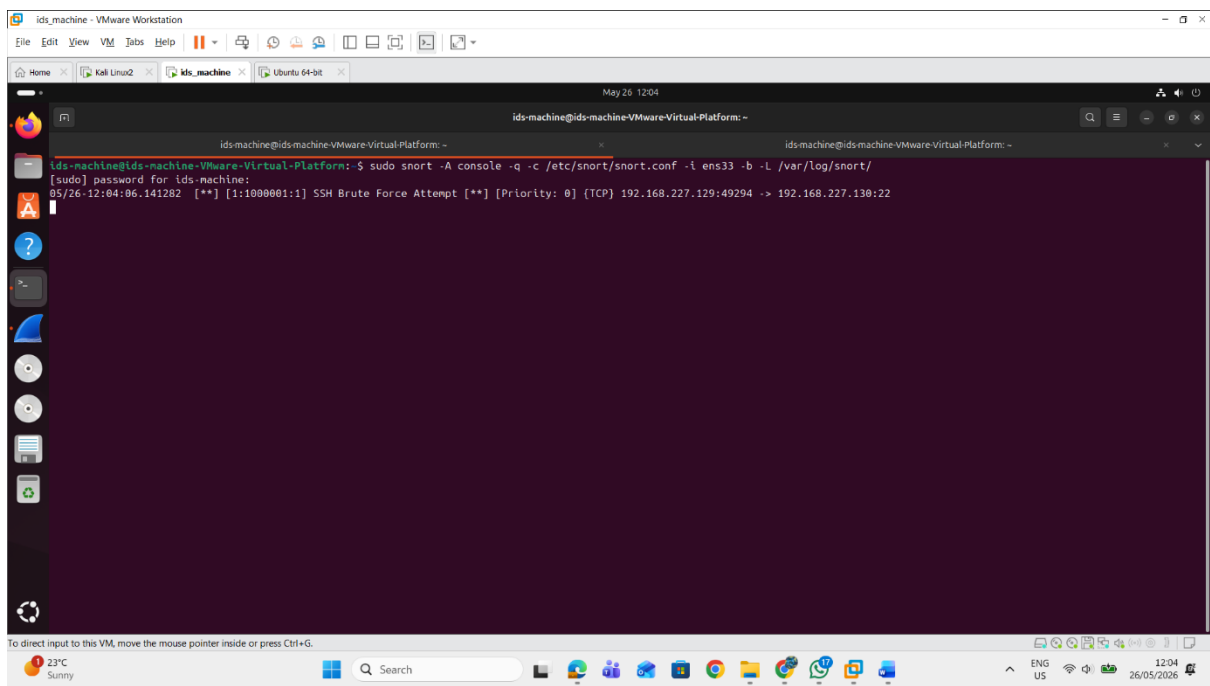
At the same time, Snort on the IDS VM detected the activity and triggered a real-time alert (SID 1000001) indicating an SSH brute-force attempt against port 22. The alert in Figure 3.3.2b confirms repeated TCP connection attempts from 192.168.227.129 to 192.168.227.130, providing evidence that the IDS successfully identified the automated attack pattern.



```
(stella@kali):~$ hydra -i ubuntu-client -P pass.txt ssh://192.168.227.138 -t 4
hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-26 12:04:05
[DATA] max 6 tasks per 1 server, overall 6 tasks, 6 login tries (1:1/pw), -> tries per task
[DATA] attacking ssh://192.168.227.138:22/
[+] ssh host 192.168.227.138 login: ubuntu-client password: root
1 of 3 target successfully completed, 1 valid password found
hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-26 12:04:12

(stella@kali):~$
```



```
ids_machine@ids-machine-VMware-Virtual-Platform:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i ens33 -b -L /var/log/snort/
[sudo] password for ids-machine:
05/26-12:04:06.141202  [**] [1:1000001:1] SSH Brute Force Attempt [**] [Priority: 0] [TCP] 192.168.227.129:49294 -> 192.168.227.130:22

ids_machine@ids-machine-VMware-Virtual-Platform:~$
```

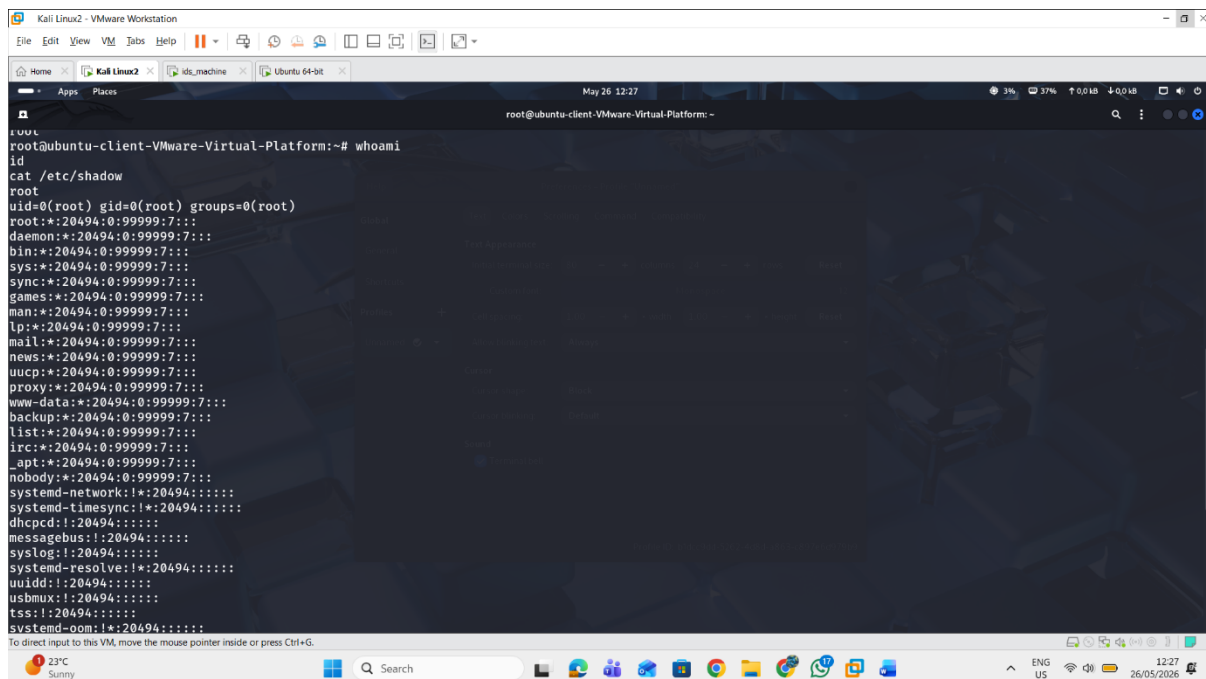
Privilege Escalation

Following initial access via SSH, privilege escalation was performed on the Client VM to determine the level of control obtained. The commands executed included `sudo su -`, `whoami`, `id`, and `cat /etc/shadow`. The `sudo su -` command was used to switch the session to the root user, resulting in a root shell (`root@ubuntu-client`). The `whoami` command confirmed the active user as `root`, while `id` verified full administrative privileges with output indicating `uid=0(root) gid=0(root)`

groups=0(root). Finally, cat /etc/shadow was executed to access the system's shadow password file, which contains secure password hashes for all user accounts.

The successful display of /etc/shadow confirms complete privilege escalation, as this file is restricted exclusively to the root user. Its contents showed that the ubuntu-client account contains a valid SHA-512 password hash, while other system accounts are locked using * or !, indicating no active credentials. In a real attack scenario, extracted hashes could be subjected to offline cracking using tools such as Hashcat or John the Ripper to further compromise the system.

Regarding IDS detection, no Snort alert was generated during this stage. Although a rule (SID 1000003) was configured to detect the keyword sudo, the activity occurred within an encrypted SSH session. As Snort operates as a signature-based NIDS, it cannot inspect encrypted payloads, preventing it from detecting the command execution. This highlights a key limitation of IDS solutions when dealing with encrypted traffic, where visibility is restricted and malicious activity must often be confirmed through endpoint-level evidence.



```
Kali Linux2 - VMware Workstation
File Edit View VM Tabs Help
Kali Linux2 x ids_machine x Ubuntu 64-bit x
May 26 12:27
root@ubuntu-client-VMware-Virtual-Platform:~# whoami
root
root@ubuntu-client-VMware-Virtual-Platform:~# id
root
root@ubuntu-client-VMware-Virtual-Platform:~# cat /etc/shadow
root:uid=0(root) gid=0(root) groups=0(root)
root:!:20494:0:99999:7:::
daemon:!:20494:0:99999:7:::
bin:!:20494:0:99999:7:::
sys:!:20494:0:99999:7:::
sync:!:20494:0:99999:7:::
games:!:20494:0:99999:7:::
man:!:20494:0:99999:7:::
lp:!:20494:0:99999:7:::
mail:!:20494:0:99999:7:::
news:!:20494:0:99999:7:::
uucp:!:20494:0:99999:7:::
proxy:!:20494:0:99999:7:::
www-data:!:20494:0:99999:7:::
backup:!:20494:0:99999:7:::
list:!:20494:0:99999:7:::
irc:!:20494:0:99999:7:::
_apt:!:20494:0:99999:7:::
nobody:!:20494:0:99999:7:::
systemd-network:!:20494:0:99999:7:::
systemd-timesync:!:20494:0:99999:7:::
dhcpcd:!:20494:0:99999:7:::
messagebus:!:20494:0:99999:7:::
syslog:!:20494:0:99999:7:::
systemd-resolve:!:20494:0:99999:7:::
uuidd:!:20494:0:99999:7:::
usbmux:!:20494:0:99999:7:::
tss:!:20494:0:99999:7:::
systemd-oom:!:20494:0:99999:7:::
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.
```

Data Exfiltration

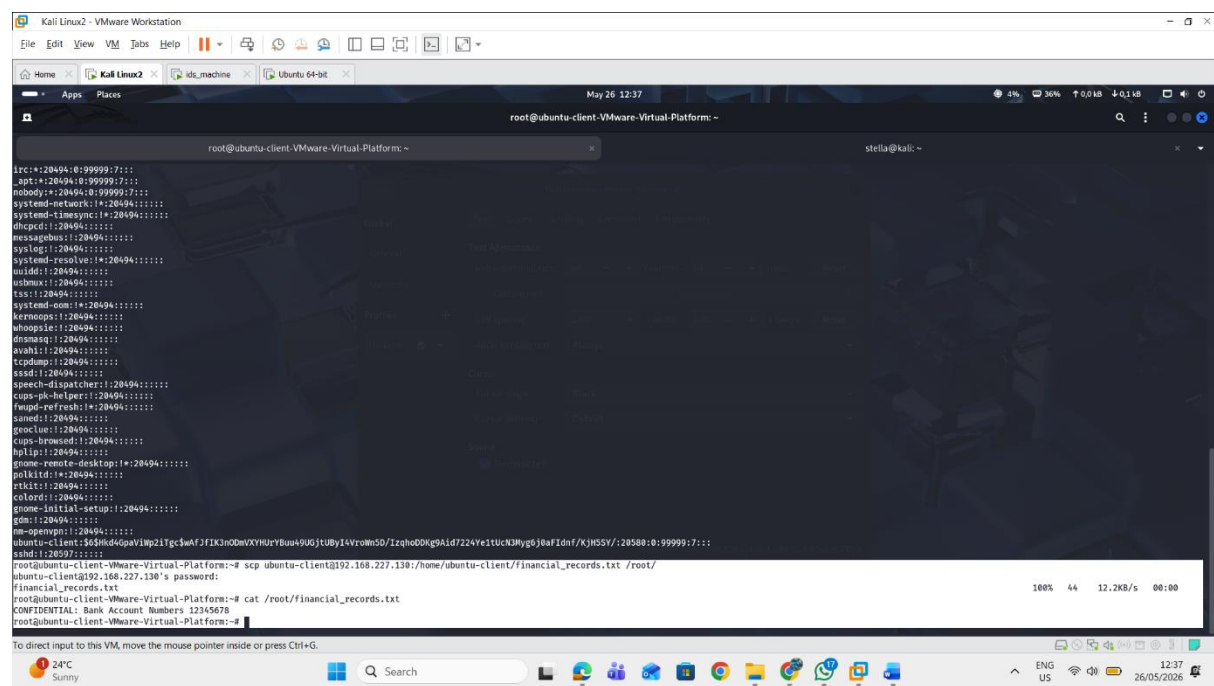
After successfully escalating privileges to root on the Client VM via the compromised SSH session, sensitive data was exfiltrated from the target system. The attacker executed the following command from the Attacker VM to securely copy the file from the victim machine:

```
scp ubuntu-client@192.168.227.130:/home/ubuntu-client/financial_records.txt /root/
```


The Secure Copy Protocol (SCP) leverages SSH to securely transfer files between hosts over an encrypted channel. In this case, the transfer was successful due to several contributing factors. First, valid credentials obtained during the brute-force phase provided authenticated SSH access to the system. Second, privilege escalation granted root-level permissions, allowing unrestricted access to protected files and directories. Third, the SSH session had already established a trusted connection between the Attacker and Client VMs, enabling seamless file transfer without additional security prompts. Finally, port 22 was open on the Client VM, allowing unrestricted inbound and outbound SSH/SCP traffic. The IDS failed to generate an alert during the file exfiltration stage due to one core reason: SCP uses an encrypted SSH tunnel.

The success of the operation was confirmed on the Attacker VM by executing `cat /root/financial_records.txt`, which displayed the extracted contents: **CONFIDENTIAL: Bank Account Numbers 12345678**. This verified that sensitive financial data had been successfully exfiltrated from the target system.

This stage completes the full attack chain, progressing from brute-force access to initial compromise, to privilege escalation, and finally data exfiltration, thereby demonstrating a realistic end-to-end intrusion scenario against an internal organisational workstation.



```
irc:*:20494:0:99999:7::
_spt:*:20494:0:99999:7::
nobody:*:20494:0:99999:7::
systemd-networkd:*:20494:0:
systemd-timesyncd:*:20494:0:
dhcpcd:*:20494:0:
messagebus:*:20494:0:
systemd-resolve:*:20494:0:
uuidd:*:20494:0:
usbmuxd:*:20494:0:
tsa:*:20494:0:
systemd-com:*:20494:0:
kernoops:*:20494:0:
whoopsie:*:20494:0:
dramasq:*:20494:0:
avahi:*:20494:0:
tcpdump:*:20494:0:
sssd:*:20494:0:
speech-dispatcher:*:20494:0:
cups-pk-helper:*:20494:0:
fwupd-refresh:*:20494:0:
saned:*:20494:0:
gnocler:*:20494:0:
cups-browsed:*:20494:0:
hplip:*:20494:0:
gnome-remote-desktop:*:20494:0:
polkitd:*:20494:0:
rtkit:*:20494:0:
colord:*:20494:0:
gnome-initial-setup:*:20494:0:
gdm:*:20494:0:
nm-openvpn:*:20494:0:
ubuntu-client:503m040pav1p21Tgc5wF3IK3n0DwVXYHUrYBus49UJtUBy14VroWmSD/Izqho0Kq9AId7224Ye1UcNjMyG3eaf1dnf/KJHSSV:/20580:0:99999:7::
sshd:*:20494:0:
root@ubuntu-client-Vmware-Virtual-Platform:~# scp ubuntu-client@192.168.227.130:/home/ubuntu-client/financial_records.txt /root/
financial_records.txt
root@ubuntu-client-Vmware-Virtual-Platform:~# cat /root/financial_records.txt
CONFIDENTIAL: Bank Account Numbers 12345678
root@ubuntu-client-Vmware-Virtual-Platform:~#
```

TCP Three-Way Handshake Analysis and Deep Packet Inspection

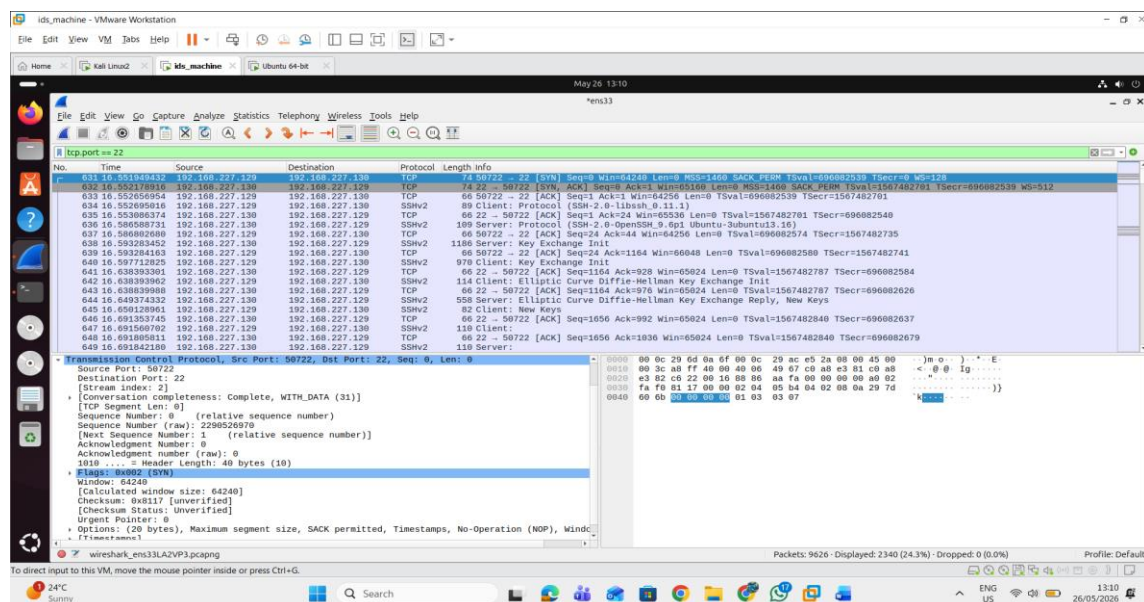
Wireshark was deployed on the IDS VM to capture and analyse network traffic during the attack simulation. Using the display filter `tcp.port == 22`, SSH traffic between the Attacker VM (192.168.227.129) and the Client VM (192.168.227.130) was isolated. The capture provides evidence of TCP handshake activity, SSH session establishment, and brute-force behaviour.

The TCP analysis confirms that each connection attempt follows a full three-way handshake. Packet 631 shows a SYN from the Attacker VM, Packet 632 a SYN-ACK from the Client VM, and Packet 633 an ACK from the Attacker VM, confirming successful TCP session establishment for each attempt.

Deep packet inspection of Packet 631 reveals key header details, including source port 50722, destination port 22 (SSH), sequence number 0, acknowledgement number 0, SYN flag (0x002), window size 64240, and a 40-byte TCP header length indicating options. These values confirm standard connection initiation behaviour. Following this, SSH negotiation identified the client as SSH-2.0-libssh_0.11.1 (automated Hydra tool) and the server as SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.16, confirming the target SSH service.

The capture also shows clear brute-force indicators, including repeated SYN packets from varying source ports, rapid connection attempts within milliseconds, and FIN/ACK responses after failed logins. A successful SSH session is also observed, confirmed by a full SSHv2 exchange, indicating valid credentials were eventually accepted.

Overall, the Wireshark analysis corroborates the Snort alert under SID 1000001, with both tools independently confirming repeated SSH connection attempts from 192.168.227.129 to 192.168.227.130, providing strong forensic evidence of an automated brute-force attack.



Incident Response

1. Disabling the SSH Service on Client VM

To prevent further unauthorised access, the SSH service on the Client VM was disabled using `systemctl stop ssh`, `systemctl stop ssh.socket`, `systemctl disable ssh`, and `systemctl disable ssh.socket`. The service stop command immediately terminates SSH, while disabling both SSH and its socket prevents automatic reactivation or startup on reboot.

This is necessary because Ubuntu uses socket-based activation, where `ssh.socket` can restart the service when a connection attempt is detected on port 22. Disabling both ensures the SSH service cannot be triggered or restored, effectively closing the remote access vector.

Validation confirmed successful containment. The Attacker VM returned Connection refused when attempting to reconnect, while the Client VM showed SSH and its socket were disabled. This confirms that SSH access was fully blocked and the attack path eliminated.

The screenshot shows a Kali Linux virtual machine with the following terminal output:

```

Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: en
Active: active (running) since Tue 2024-05-26 11:59:03 CAT; 15s ago
    Triggers: @ssh.socket
           local: man:ssh(6)
           man:ssh-config(5)
Process: 6912 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
Main PID: 4917 (sshd)
Tasks: 1 (limit: 4941)
Memory: 2.1M (peak: 2.4M)
CPU: 46mu
CGroup: /system.slice/ssh.service
        └─sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups

May 26 11:59:03 ubuntu-client-Vmware-Virtual-Platform systemd[1]: Starting ssh:
May 26 11:59:03 ubuntu-client-Vmware-Virtual-Platform sshd[4917]: Server listen
May 26 11:59:03 ubuntu-client-Vmware-Virtual-Platform sshd[4917]: Server listen
May 26 11:59:03 ubuntu-client-Vmware-Virtual-Platform systemd[1]: Started ssh:
[Ubuntu 22.04 LTS]
ubuntu-client@ubuntu-client-Vmware-Virtual-Platform:~$ sudo systemctl stop ssh
[sudo] password for ubuntu-client:
Warning: The unit file, source configuration file or drop-ins of ssh.service changed on disk. Run 'systemctl daemon-reload' to reload units.
Stopping 'ssh.service', but its triggering units are still active:
ssh.socket
ubuntu-client@ubuntu-client-Vmware-Virtual-Platform:~$ sudo systemctl stop ssh
sudo systemctl stop ssh.socket
sudo systemctl disable ssh
sudo systemctl disable ssh.socket
Warning: The unit file, source configuration file or drop-ins of ssh.service changed on disk. Run 'systemctl daemon-reload' to reload units.
Stopping 'ssh.service', but its triggering units are still active:
ssh.socket
Warning: The unit file, source configuration file or drop-ins of ssh.socket changed on disk. Run 'systemctl daemon-reload' to reload units.
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Removed '/etc/systemd/system/sockets.target.wants/ssh.socket'.
Removed '/etc/systemd/system/ssh.service.requires/ssh.socket'.
ubuntu-client@ubuntu-client-Vmware-Virtual-Platform:~$

```

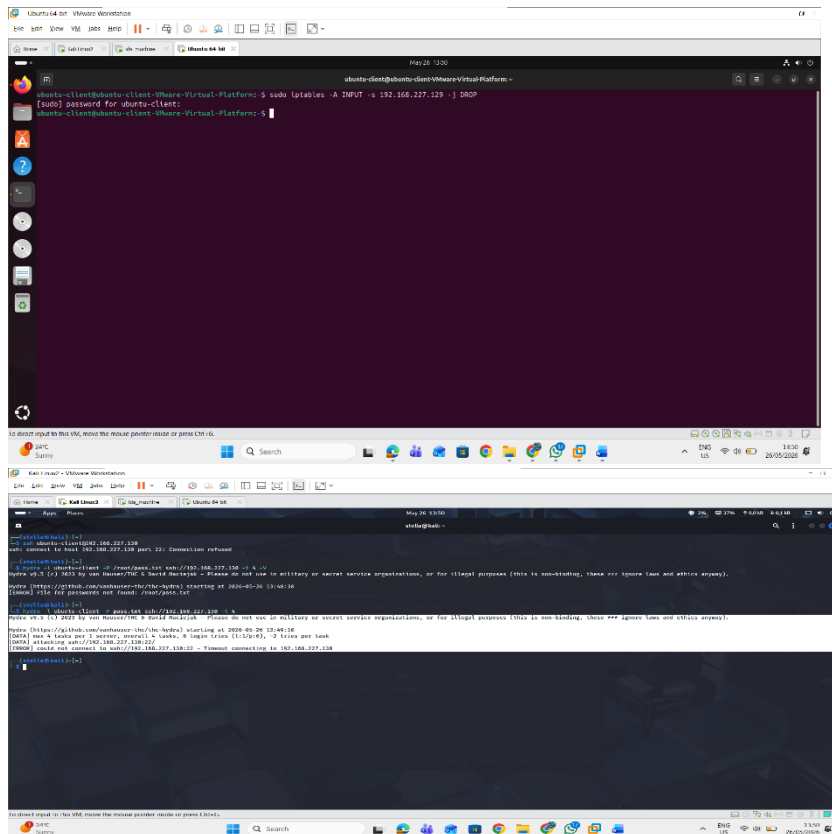
To direct input to the VM, move the mouse pointer inside or press Ctrl-G.

2. Blocking Attacker IP via iptables Firewall Rule

To further secure the Client VM, a host-based firewall rule was implemented to block all incoming traffic from the Attacker VM. This was achieved using the command `sudo iptables -A INPUT -s 192.168.227.129 -j DROP`, which appends a rule to the INPUT chain to silently drop all packets originating from the attacker's IP address. In this rule, iptables is the Linux firewall utility, -A INPUT applies the rule to all incoming traffic, -s 192.168.227.129 specifies the source IP of the Attacker VM, and -j DROP ensures that matching packets are discarded without any response.

This method provides stronger protection than simply disabling SSH because it operates at the network layer rather than the service layer. As a result, all traffic from the attacker is blocked regardless of which service or port is targeted. Even if SSH or other services were later re-enabled, the attacker would still be unable to reach the system. The use of DROP instead of REJECT is significant, as DROP silently discards packets without notifying the sender, whereas REJECT immediately informs the attacker that the connection was refused.

Post-implementation testing confirmed that the rule was effective. When the Attacker VM attempted another brute-force attack using Hydra, it returned a timeout error: [ERROR] could not connect to ssh://192.168.227.130:22 - Timeout connecting. This confirms that all traffic from 192.168.227.129 was successfully blocked at the firewall level, effectively neutralising further intrusion attempts against the Client VM.



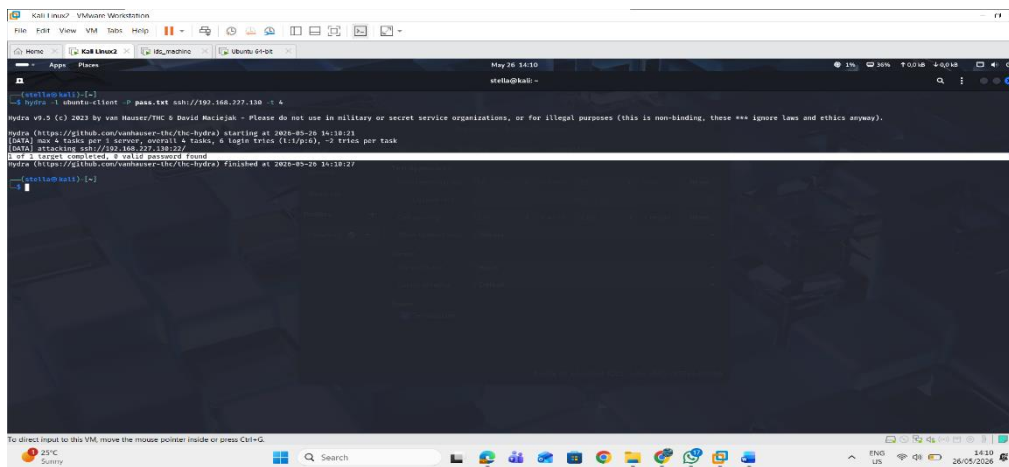
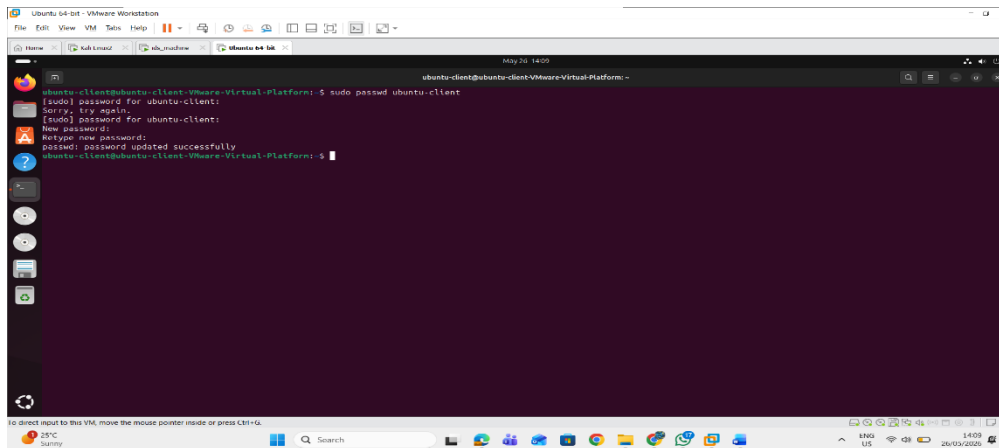
3. Enforcing a Strong Password Policy

To prevent further brute-force attacks, the weak password on the ubuntu-client account was replaced using `sudo passwd ubuntu-client`, updating the credential stored in `/etc/shadow`. The new password set was `N@m!b1a$ecure2026`, replacing the previously weak password `root`.

The new password is significantly stronger due to its 16-character length and inclusion of uppercase letters, lowercase letters, numbers, and special characters. Unlike the old password, which is common in attack wordlists, the new password is not present in standard dictionaries, making it resistant to dictionary-based attacks.

When Hydra is rerun with the same wordlist, all attempts fail and return 0 valid passwords found, confirming that no match exists. This is because the password is not in the dictionary, and its complexity makes brute-force guessing computationally impractical.

Overall, enforcing a strong password policy removes the original vulnerability caused by weak credentials and ensures the system is no longer susceptible to automated brute-force attacks.



Conclusion

This project successfully demonstrated the deployment and operational effectiveness of a Network Intrusion Detection System within a simulated financial organisation environment. Through the execution of a multi-stage attack chain SSH brute-force, privilege escalation, and data exfiltration the project provided practical evidence of how poorly secured systems can be compromised and how intrusion detection tools respond in real time.

Snort successfully detected the brute-force attack, generating real-time alerts that were corroborated by Wireshark packet captures confirming the TCP three-way handshake and automated attack patterns. However, the project also exposed a significant limitation of signature-based NIDS tools the inability to inspect encrypted traffic as both the privilege escalation and data exfiltration stages conducted over SSH produced no alerts. This highlights the need for a layered security approach that goes beyond signature-based detection alone.

The incident response phase demonstrated three effective containment measures disabling SSH, blocking the attacker IP via iptables, and enforcing a strong password

policy each validated through post-incident testing confirming the attack could no longer be executed under the hardened configuration.

In conclusion, this project reinforced that effective cybersecurity requires a combination of proactive monitoring, strong access controls, and a well-practised incident response capability. The hands-on simulation provided valuable insight into real-world attack techniques and the defensive measures available to protect sensitive financial systems against unauthorised access.

References

DeepStrike. (2025). *Zero-day exploit statistics: The 2025 threat report for defenders*. Deepstrike Cybersecurity. <https://deepstrike.io/blog/zero-day-exploit-statistics-2025>

IBM. (2025). *Cost of a data breach report 2025*. IBM Corporation. <https://www.ibm.com/reports/data-breach>

National Institute of Standards and Technology. (2012). *Guide to intrusion detection and prevention systems (IDPS)* (Special Publication 800-94 Rev. 1). U.S. Department of Commerce. <https://csrc.nist.gov/publications/detail/sp/800-94/rev-1/final>